

Control System Design with MATLAB and Simulink

— Complete Notes

Learning path: 5 courses · ~3.5 hours total **Source:** <https://matlabacademy.mathworks.com/details/control-system-design-with-matlab-and-simulink/lpmlslcsd> **Recommended prerequisites:** MATLAB Onramp, Simulink Onramp **Prepared for:** HACKSIMUL8 2026 — PES University, 2 Sept 2026

Access note: on the live site every course in this path is marked "*This course is locked — Available through the Online Training Suite.*" The path structure below is the real one read from the course page; the content is rebuilt from MathWorks documentation for Control System Toolbox and Simulink Control Design.

Path structure

#	Course	Time	Stated objective
1	Control System Modeling Essentials	1 h	Create control system objects in MATLAB and Simulink and simulate their behaviours
2	Linearization of Nonlinear Systems	0.5 h	Linearise nonlinear systems at the appropriate operating points
3	Control System Analysis Techniques	1 h	Analyse control systems to explore their critical properties
4	PID Control Techniques	0.5 h	Design PID controllers for real-world systems
5	Classical Controller Design Techniques	0.5 h	Controller design using classical methods

This is the most important file for HACKSIMUL8 — the organisers named Control System Toolbox explicitly.

Course 1 — Control System Modeling Essentials

The three model representations

```
% Transfer function:  $G(s) = (s+2) / (s^2 + 3s + 5)$ 
G = tf([1 2], [1 3 5]);

% Zero-pole-gain form
G = zpk(-2, [-1+2i, -1-2i], 1);

% State space
A = [0 1; -5 -3]; B = [0; 1]; C = [1 0]; D = 0;
G = ss(A, B, C, D);

% Frequency response data
G = frd(response, frequencies);
```

Convert freely between them: `tf(sys)`, `zpk(sys)`, `ss(sys)`, `frd(sys, w)`.

Which to use:

- `tf` — quickest to type; how textbook problems are stated.
- `zpk` — makes poles and zeros explicit, so stability is visible at a glance.
- `ss` — required for MIMO work, modern control (LQR, observers), and what `linearize` returns.

Building the classic plants fast

```
s = tf('s'); % then write transfer functions algebraically
G = 1/(s^2 + 2*s + 1); % much more readable than coefficient vectors

% Standard second-order system
wn = 5; zeta = 0.4;
G = wn^2 / (s^2 + 2*zeta*wn*s + wn^2);

% First-order lag with time constant tau and DC gain K
K = 2; tau = 0.5;
G = K/(tau*s + 1);

% Add a pure time delay of 0.2 s
G.InputDelay = 0.2;
```

Discrete-time models

```
Gd = tf([1 -0.5], [1 -1.2 0.32], 0.1); % last argument is Ts
Gd = c2d(G, 0.1, 'zoh'); % 'zoh' | 'tustin' | 'matched'
Gc = d2c(Gd);
```

Interconnecting systems

```
series(G1, G2) % or simply G2*G1
parallel(G1, G2) % or G1 + G2
feedback(G, H) % negative feedback by default
feedback(G, H, +1) % positive feedback
feedback(C*G, 1) % closed loop, unity feedback <-- the one you use constantly
```

The four transfer functions of a feedback loop — know these, judges ask:

```
L = C*G; % open loop
T = feedback(L, 1); % complementary sensitivity: reference -> output
S = feedback(1, L); % sensitivity: disturbance -> output; S + T = 1
U = feedback(C, G); % reference -> control effort (checks actuator saturation)
```

Simulating behaviour

```
step(G) % unit step response
impz(G)
lsim(G, u, t) % arbitrary input
initial(sys_ss, x0) % state-space initial-condition response

[y, t] = step(G); % capture data instead of plotting
stepinfo(G) % RiseTime, SettlingTime, Overshoot, Peak, PeakTime
dcgain(G) % steady-state gain
pole(G), zero(G)
damp(G) % natural frequency and damping ratio of each pole
```

`stepinfo` is the fastest way to produce hard performance numbers for a slide.

Control system objects in Simulink

- **Transfer Fcn, State-Space, Zero-Pole** blocks accept workspace variables directly — put `G.num{1}` and `G.den{1}`, or better, use an **LTI System** block that takes the object `G` whole.
- Keep the plant as a MATLAB object and reference it from the model. Then you can analyse in MATLAB and simulate in Simulink with a guarantee that they are the same plant.

Course 2 — Linearization of Nonlinear Systems

Nearly all classical design theory (Bode, root locus, margins, PID tuning) assumes a **linear** model. Real plants are not linear. Linearisation bridges the two.

The idea

Take a nonlinear system $\dot{x} = f(x, u)$, pick an **operating point** (x_0, u_0) , and approximate with the Jacobians:

$$\dot{x} \approx A(x - x_0) + B(u - u_0), \quad A = \left. \frac{df}{dx} \right|_o, \quad B = \left. \frac{df}{du} \right|_o$$

The result is valid **near that operating point only**. State the operating point when you present results — it is the first thing a knowledgeable judge will probe.

Choosing the operating point

An **equilibrium (trim) point** is where the derivatives are zero, so the system is at steady state. That is normally what you want.

```
% Trim the model to an equilibrium point
opspec = operspec('mymodel');
op = findop('mymodel', opspec);

% Or take a snapshot from a simulation at t = 10 s
op = findop('mymodel', 10);
```

Defining what to linearise: analysis points

```
io(1) = linio('watertank/PID Controller', 1, 'input');
io(2) = linio('watertank/Water-Tank System', 1, 'openoutput');
```

Analysis-point types:

- **input** (input perturbation) — where the signal is injected
- **output** (output measurement) — where the response is measured
- **openoutput** (open-loop output) — an output measurement *followed by a loop opening*, which removes the effect of the feedback signal on the linearisation

Getting **openoutput** versus **output** wrong is the most common linearisation mistake. Use **openoutput** when you want the **open-loop** $L = C \cdot G$ for margin analysis; use **output** when you want the closed-loop response.

Inspect or reuse existing I/O settings with `getlinio('mymodel')`.

Running the linearisation

```
linsys = linearize('mymodel', io);           % at the model initial condition
linsys = linearize('mymodel', op, io);       % at a trimmed operating point

bode(linsys)
step(linsys)
pole(linsys)
```

Two routes exist: programmatic (`linearize`) and the **Model Linearizer** app, which is the GUI equivalent and easier under time pressure.

Practical warnings

- **Blocks that do not linearise well:** Saturation, Dead Zone, Relay, Quantizer, Coulomb friction, and Stateflow charts all linearise to either a constant gain or zero. If your linearised model comes back as all zeros, a discontinuous block on the path is the usual culprit.
- Check with `linsys.A` — an empty or zero `A` matrix means the linearisation found no dynamics.
- Linearise at several operating points across the range and design for the worst case. This is *gain scheduling* in embryo, and it impresses judges.

Course 3 — Control System Analysis Techniques

Time-domain analysis

```
S = stepinfo(T)
% RiseTime, SettlingTime, SettlingMin, SettlingMax,
% Overshoot, Undershoot, Peak, PeakTime
```

For a standard second-order system with damping `zeta` and natural frequency `wn`:

Quantity	Relation
Overshoot	<code>%OS = 100*exp(-pi*zeta/sqrt(1-zeta^2))</code>
Settling time (2 %)	<code>Ts ≈ 4/(zeta*wn)</code>
Rise time (approx)	<code>Tr ≈ 1.8/wn</code>
Damped frequency	<code>wd = wn*sqrt(1-zeta^2)</code>

Useful anchors: `zeta = 0.707` gives about 4.3 % overshoot; `zeta = 0.5` gives about 16 %; `zeta = 1` is critically damped with no overshoot.

Stability

```
pole(T)
isstable(T)
pzmap(T)
```

- **Continuous time:** stable if every pole has a negative real part (left half plane).
- **Discrete time:** stable if every pole lies inside the unit circle, `abs(pole) < 1`.

Frequency-domain analysis

```
bode(L), bodeplot(L)
nyquist(L)
nichols(L)
sigma(T) % singular values, for MIMO
[Gm, Pm, Wcg, Wcp] = margin(L)
margin(L) % plots and annotates the margins
```

Interpreting `margin` (from the MathWorks reference):

Output	Meaning	Unit
<code>Gm</code>	gain change needed to make the loop gain unity at the frequency where phase is -180°	absolute , not dB
<code>Pm</code>	difference between the phase response and -180° where loop gain is 1	degrees
<code>Wcg</code>	frequency of the -180° phase crossing	rad/TimeUnit
<code>Wcp</code>	frequency of the 0 dB gain crossing	rad/TimeUnit

`Gm` comes back in absolute units. Convert with `Gm_dB = 20*log10(Gm)`. Quoting a gain margin of "2" when you meant 6 dB is an easy way to lose credibility.

Targets to design toward:

- Gain margin ≥ 6 dB (absolute ≈ 2)
- Phase margin 45° – 60° — 60° gives a well-damped response, 45° is the usual minimum
- Bandwidth `wc` sets speed; response time is roughly `1/wc`

`bandwidth(T)` returns the closed-loop bandwidth.

Root locus

```
rlocus(G)
rlocfind(G) % click the plot to read off the gain
sgrid(zeta, wn) % overlay damping and natural-frequency contours
```

Root locus shows how closed-loop poles migrate as a single gain `K` sweeps from 0 to ∞ . Use it to pick `K` for a target damping ratio: overlay `sgrid(0.707, [])` and choose where the locus crosses that ray.

The interactive tools

```
controlSystemDesigner(G)    % classic tuning: root locus + Bode + step, live
pidTuner(G, 'PID')          % interactive slider for response time and transient behaviour
linearSystemAnalyzer(T)     % all response plots in one window
```

`controlSystemDesigner` is worth knowing on the day: drag a pole, watch the step response update. It makes for a strong live demo.

Course 4 — PID Control Techniques

What each term does

Term	Effect	Cost
P — proportional	reduces rise time and steady-state error	never eliminates SS error; too much causes oscillation
I — integral	eliminates steady-state error	adds phase lag, reduces stability, causes windup
D — derivative	adds damping, reduces overshoot, improves stability	amplifies measurement noise

Parallel form:

$$u(t) = K_p \cdot e + K_i \int e \, dt + K_d \cdot de/dt$$

Creating controllers

```
C = pid(Kp, Ki, Kd)           % parallel form
C = pid(Kp, Ki, Kd, Tf)       % with derivative filter (use this)
C = pidstd(Kp, Ti, Td)        % standard/ideal form
C = pid2(...)                 % 2-DOF, with setpoint weights b and c
```

Always use a derivative filter. Pure `Kd*s` is not realisable and amplifies noise; the filtered form is `Kd*s/(Tf*s + 1)`. Set `Tf` around `Kd/(10*Kp)`.

Automatic tuning — `pidtune`

```
C = pidtune(sys, type)
C = pidtune(sys, C0)
C = pidtune(sys, wc)
C = pidtune(sys, opts)
[C, info] = pidtune(____)
```

`info` returns:

- `Stable` — whether the closed loop is stable
- `CrossoverFrequency` — the first 0 dB gain crossover frequency, in rad/TimeUnit
- `PhaseMargin` — in degrees

Examples straight from the reference page:

```
sys = zpk([], [-1 -1 -1], 1);
[C_pi, info] = pidtune(sys, 'PI');

% Ask for a faster loop by specifying the target crossover frequency
[C_fast, info] = pidtune(sys, 'PI', 1.0);

% Standard form
C0 = pidstd(1, 1, 1);
C = pidtune(sys, C0);

% Discrete time with a chosen integration formula
sysd = c2d(tf([1 1], [1 5 6]), 0.1);
C0 = pid(1, 1, 'Ts', 0.1, 'IFormula', 'BackwardEuler');
C = pidtune(sysd, C0);
```

The `wc` argument is your speed dial. It sets a target for the 0 dB gain crossover frequency, which approximately fixes the control bandwidth: response time is roughly $1/wc$. Raise `wc` for a faster response, lower it for more stability.

Shape the design further with `pidtuneOptions`, which supports a target **phase margin** and a **design focus** (reference tracking versus disturbance rejection).

Controller `type` strings: `'P'`, `'I'`, `'PI'`, `'PD'`, `'PDF'`, `'PID'`, `'PIDF'`. The `F` variants include the derivative filter — prefer them.

Manual tuning, when `pidtune` is not available

Ziegler–Nichols, ultimate-gain method:

1. Set `Ki = Kd = 0`. Raise `Kp` until the output oscillates with constant amplitude.
2. Record that gain as `Ku` and the oscillation period as `Tu`.
3. Then:

Controller	Kp	Ki	Kd
P	$0.5 * K_u$	—	—
PI	$0.45 * K_u$	$0.54 * K_u / T_u$	—
PID	$0.6 * K_u$	$1.2 * K_u / T_u$	$0.075 * K_u * T_u$

Ziegler–Nichols is aggressive, typically giving 25 % overshoot. Treat it as a starting point and back **Kp** off.

Practical loop: raise **Kp** until the response is fast but oscillatory → add **Kd** to damp it → add **Ki** last, only as much as is needed to kill the steady-state error.

Integrator windup — the bug that will bite you

When the actuator saturates, the integral term keeps accumulating even though more control effort cannot be delivered. The output then overshoots badly while the integrator unwinds.

Fixes:

- Use the Simulink **PID Controller** block and enable **Limit output** plus **Anti-windup method** (**back-calculation** or **clamping**).
- Or use an Integrator block with **Limit output** enabled.
- Always model the real actuator limit with a **Saturation** block. A control demo without actuator limits is not credible.

The Simulink PID Controller block

The block you should actually use, rather than assembling P, I and D by hand:

- **Form:** Parallel or Ideal; **Time domain:** Continuous or Discrete
- **Tune...** button opens PID Tuner, which linearises the plant around the operating point and tunes automatically — this is the fastest path from model to working controller on the day
- Built-in **output saturation** and **anti-windup**
- Optional **derivative filter** coefficient **N**
- **Setpoint weighting** for 2-DOF control, which cuts the derivative kick on step changes

Course 5 — Classical Controller Design Techniques

Lead compensator

$$C(s) = K_c * (s + z)/(s + p) \text{ with } z < p.$$

Adds **positive phase** near the crossover, so it improves phase margin and speeds the response. It is the frequency-domain cousin of derivative action.

```
% Add phi degrees of phase at frequency wm
alpha = (1 - sind(phi)) / (1 + sind(phi));
T      = 1 / (wm * sqrt(alpha));
C      = (T*s + 1) / (alpha*T*s + 1);
```

Maximum phase lift occurs at $\omega_m = 1/(T\sqrt{\alpha})$. A single lead section gives at most about 60°; cascade two if you need more.

Lag compensator

$C(s) = K_c * (s + z)/(s + p)$ with $z > p$.

Raises the low-frequency gain, so it reduces **steady-state error** without much affecting crossover. It is the frequency-domain cousin of integral action. Place the zero and pole well below the crossover frequency so the phase lag does not eat your margin.

Lead-lag

Cascade both: lag for steady-state accuracy, lead for transient response and margin.

Design procedure using Bode

1. Plot `bode(G)` and read the current `margin(G)`.
2. Decide the target crossover `wc` from the required speed (`response time ≈ 1/wc`).
3. Work out how much phase you must add at `wc` to reach a 45°–60° phase margin.
4. Design a lead section to supply it; centre `wm` on `wc`.
5. Set the gain `Kc` so the magnitude actually crosses 0 dB at `wc`.
6. Verify with `margin(C*G)` and `step(feedback(C*G, 1))`.

Root-locus design procedure

1. `rlocus(G)` and `sgrid(zeta, wn)` for the target region.
2. If the locus already passes through the target, a simple gain suffices — read it off with `rlocfind`.
3. If it does not, add a compensator zero to pull the locus left (faster, better damped), or a pole to push it right.
4. Verify with a step response.

Steady-state error and system type

System **type** = number of pure integrators (poles at the origin) in the open loop `L`.

Type	Step error	Ramp error	Parabola error
0	$1/(1+K_p)$	∞	∞
1	0	$1/K_v$	∞
2	0	0	$1/K_a$

with $K_p = \lim_{s \rightarrow 0} L(s)$, $K_v = \lim_{s \rightarrow 0} sL(s)$, $K_a = \lim_{s \rightarrow 0} s^2L(s)$.

Practical consequence: to remove steady-state error to a step, the loop needs at least one integrator. If the plant has none, the controller must supply it — which is precisely what the **I** term in PID does.

One-page hackathon workflow

```
%% 1. Plant
s = tf('s');
G = 1/(s^2 + 2*s + 1);          % or linearize('mymodel', io)

%% 2. Look before designing
figure; step(G); figure; bode(G); margin(G)
pole(G), isstable(G)

%% 3. Design
[C, info] = pidtune(G, 'PIDF');
info
% Stable, CrossoverFrequency, PhaseMargin

%% 4. Close the loop and analyse
L = C*G;
T = feedback(L, 1);
S = feedback(1, L);
figure; step(T); stepinfo(T)
figure; margin(L)

%% 5. Faster? Re-tune against a target crossover
[C2, info2] = pidtune(G, 'PIDF', 2*info.CrossoverFrequency);

%% 6. Check control effort against the actuator limit
U = feedback(C, G);
figure; step(U)                % does it exceed what the actuator can deliver?
```

Numbers to state when you present: phase margin (deg), gain margin (dB), bandwidth or crossover (rad/s), settling time, percent overshoot, steady-state error, and peak control effort. Those seven cover almost every question a judge will ask.

Sanity checks before you demo

- Does the closed-loop step response settle at the reference value? If not, you have a steady-state error and probably need integral action.
- Is there a **Saturation** block modelling the actuator? Without one your controller is cheating.
- Is anti-windup enabled if you saturate and integrate?
- Are the plots labelled, with units on the axes?
- Can you state the operating point you linearised around?